

Retrieval & Knowledge Base Architecture Guideline

Purpose

This document records the architectural decisions for the Retrieval subsystem and the Knowledge Base layer of the Strategic Knowledge Engine. The goal is to:

- Preserve the design rationale behind key decisions
- Provide guidance for further development
- Ensure architectural consistency during future extensions
- Serve as a reference document within the project context

The document focuses on two main components:

1. Knowledge Base design
 2. Retrieval architecture
-

1. Architectural Principles

The retrieval system is designed according to the following principles:

1.1 Separation of Responsibilities

The architecture separates the following concerns:

User Intent → Retrieval Strategy → Knowledge Retrieval → Knowledge Interpretation → Answer Generation

This separation prevents logic from being duplicated across modules and improves explainability.

1.2 Retrieval Is Not Generation

The Retrieval Engine is responsible only for:

- locating relevant knowledge
- ranking knowledge sources
- constructing a context for the LLM

It does **not**:

- determine user intent
- determine advisory role
- generate answers

Those responsibilities belong to other components.

1.3 Governance-Aware Retrieval

The retrieval system must respect the governance model of the knowledge system.

This means:

- regulatory sources must be prioritized
- marketing material must be downgraded
- document trustworthiness must influence ranking

Governance signals are therefore integrated directly into the ranking process.

2. System Context

The retrieval system operates within the following pipeline:

Query → Intent Engine → Role Engine → Retrieval Engine → Perspective Layer → Prompt Governance → LLM

Retrieval therefore receives structured query context rather than raw user queries.

3. Query Context

Retrieval receives a structured context object created by the pipeline.

Example structure:

QueryContext

Fields:

- query
- interaction_intent
- content_intent
- role_name
- role_description

Retrieval primarily uses:

- query
- content_intent

The interaction intent and role are used later during answer generation.

4. Knowledge Base Design

Decision

The system uses a **KnowledgeBase object combined with stateless retrieval functions**.

This hybrid architecture was selected because it combines:

- architectural clarity
- testability
- extensibility

KnowledgeBase Responsibilities

The KnowledgeBase object acts as a container for all structured knowledge artifacts.

Typical contents:

- chunks
- embeddings
- document registry
- vector index

Example conceptual structure:

KnowledgeBase

- chunks
- embeddings
- registry

The KnowledgeBase is part of the **Knowledge Base layer** of the architecture and is populated by the Knowledge Construction pipeline.

Retrieval Engine Responsibilities

The Retrieval Engine remains stateless.

It operates on artifacts provided by the KnowledgeBase but does not own them.

Example interface:

```
retrieval_engine(context, embeddings, chunks)
```

This preserves testability and modularity.

5. Retrieval Strategy

The system uses a **multi-stage retrieval pipeline** designed to balance semantic relevance and governance constraints.

The pipeline consists of five steps.

Step 1 — Candidate Retrieval

Purpose:

Generate a broad pool of semantically relevant candidates.

Process:

- Compute query embedding
- Perform vector similarity search
- Retrieve top candidate chunks

Parameter:

Top-K Candidate Retrieval

Recommended value:

Top 100 chunks

Rationale:

A large candidate pool improves recall and ensures that governance ranking has sufficient data.

Step 2 — Governance Scoring

Purpose:

Adjust ranking based on trustworthiness of sources.

Governance factors:

- document_type
- origin
- bias_level
- confidence

Scoring formula:

$\text{final_score} = \text{similarity} \times \text{governance_weight} \times \text{intent_modifier}$

This ensures that:

- regulatory documents receive priority
 - vendor marketing material is downgraded
-

Step 3 — Document Grouping

Purpose:

Group candidate chunks by their source document.

Rationale:

Chunks alone often lack context.

Document grouping ensures that retrieval remains document-centric rather than fragment-centric.

Output structure:

Document → list of candidate chunks

Step 4 — Document Ranking

Purpose:

Rank documents based on the strongest supporting evidence.

Recommended scoring rule:

$\text{Document score} = \max(\text{chunk_score})$

Rationale:

Using the maximum chunk score ensures that documents containing highly relevant passages are prioritized.

Step 5 — Context Selection

Purpose:

Construct the final context for the LLM.

Strategy:

- Select top documents
- Select best chunks within those documents

Recommended parameters:

Top documents: 3 Chunks per document: 2

Final context size:

6 chunks

This structure balances diversity of sources with sufficient context depth.

6. Retrieval Pipeline Summary

Final retrieval process:

Query → Candidate Retrieval → Governance Scoring → Document Grouping → Document Ranking → Context Selection → RetrievalResult

7. Retrieval Result Structure

The Retrieval Engine returns a structured result object.

Example:

RetrievalResult

Fields:

- query
- documents
- chunks

This structure supports:

- explainability
 - debugging
 - downstream processing
-

8. Explainability Goals

The retrieval system must support traceability.

For each retrieved chunk the system should be able to explain:

- similarity score
- governance weight
- final ranking score
- originating document

This enables the system to justify why specific knowledge sources were selected.

9. Future Extensions

The architecture supports future improvements without redesigning the pipeline.

Possible extensions include:

Hybrid Retrieval

- BM25 + vector search

Reranking Models

- cross-encoder reranking

Metadata Filtering

- optional pre-filtering for special use cases

Knowledge Graph Integration

- linking documents and concepts

These extensions can be integrated into the existing pipeline.

10. Summary

Key architectural decisions:

Knowledge Base

- KnowledgeBase object stores knowledge artifacts
- Retrieval functions remain stateless

Retrieval

- multi-stage pipeline
- governance-aware ranking
- document-level reasoning

These design choices ensure that the Knowledge Engine remains:

- explainable
- modular
- extensible
- governance-compliant

This document serves as the guideline for implementing and evolving the retrieval subsystem of the Strategic Knowledge Engine.